

Guardrail Results Summary

Sep 29, 2026 · @Sumit Goyal

Results across all four guardrail techniques tested in this repo: prompt-only baselines (Task 7/8/9), prompt + LLM judge, prompt + deterministic rules (hybrid), and the full three-layer cascade (Task 27).

Task 7/8/9 - prompt-only baselines

Task 9's structured, context-aware prompt is the only one of the three that changes model behavior at all; Task 7 and Task 8 allow almost every record regardless of ground truth.

Task	Prompt version	Accuracy	Attack recall	Precision	F1	FPR
Task 7	v7_baseline	0.500	0.000	0.000	0.000	0.000
Task 8	v8_basic_security	0.511	0.025	0.833	0.049	0.005
Task 9	v9_context_aware	0.744	0.967	0.668	0.790	0.479

1,200-record run, `destination_blind` view, `llama3.2:1b`. Task 7 (undefended baseline) and Task 8 (general security-principle prompt) allow essentially everything - Task 8's one added sentence of guidance moves attack recall from 0.000 to only 0.025. Task 9's structured procedure is the first prompt that actually changes decisions: it catches 96.7% of attacks, but at the cost of a 47.9% false-positive rate - it blocks nearly half of legitimate requests to get there. A smaller 30-record pilot (in `prompt/README.md`) shows the same pattern at a smaller scale (Task 9: 0.767 accuracy, 0.933 recall, 0.400 FPR).

Prompt + LLM judge

Plain prompting scores best; every added textual defense (delimiting, sanitizing, datamarking, provenance tagging, full defense) trades recall away instead of adding it.

Method	Accuracy	Attack recall	Benign allow rate
plain	0.945	0.930	0.960
delimited	0.865	0.790	0.940

Method	Accuracy	Attack recall	Benign allow rate
sanitized	0.870	0.800	0.940
datamarked	0.860	0.780	0.940
provenance	0.855	0.770	0.940
full_defense	0.830	0.720	0.940

200 Task 7 traces judged by Gemini (`gemini-3.5-flash-lite`) under 6 prompting methods (1,200 judge calls total). Each additional layer of prompt-side defense text made the judge *more* cautious in a way that cost recall without buying accuracy back - `full_defense` , the most heavily defended prompt, is also the worst performer.

Prompt + deterministic (hybrid)

Combining a prompt agent with a deterministic rule set never beat the better of the two alone - it only ever matched it or made it worse.

Solo baselines:

Guardrail	Accuracy	Recall	Precision	FPR
v7_baseline	0.500	0.000	0.000	0.000
v8_basic_security	0.510	0.020	1.000	0.000
v9_context_aware	0.650	0.870	0.604	0.570
basic_rules	0.500	0.000	0.000	0.000
provenance_rules	1.000	1.000	1.000	0.000
ci_norm	1.000	1.000	1.000	0.000

All 9 combinations (most-restrictive-wins):

Combination	Accuracy	Recall	Precision	FPR
v7 + basic_rules	0.500	0.000	0.000	0.000
v7 + provenance_rules	1.000	1.000	1.000	0.000

Combination	Accuracy	Recall	Precision	FPR
v7 + ci_norm	1.000	1.000	1.000	0.000
v8 + basic_rules	0.510	0.020	1.000	0.000
v8 + provenance_rules	1.000	1.000	1.000	0.000
v8 + ci_norm	1.000	1.000	1.000	0.000
v9 + basic_rules	0.650	0.870	0.604	0.570
v9 + provenance_rules	0.715	1.000	0.637	0.570
v9 + ci_norm	0.715	1.000	0.637	0.570

200-record `destination_blind` pilot, `llama3.2:1b`. The best result in the whole 15-row matrix is a deterministic rule set **alone** (`provenance_rules` or `ci_norm` , both 1.000 accuracy / 0 FPR) - because the combiner only ever escalates a decision, never softens it, pairing a strong rule with a weak prompt reproduces the rule's own score at best, and pairing a strong rule with an over-blocking prompt (`v9`) drags accuracy down to 0.715 by inheriting `v9` 's false positives on top of a rule set that had already caught every attack on its own.

Full cascade (Task 27): Deterministic -> LLM judge -> ML

The full 7,200-record run scores 0.917 accuracy / 0.889 recall - but Gemini (Layer 2) resolved 0 of the 5,200 records it was sent, so this is really a Layer 1 + Layer 3 result, not a true three-layer one.

Run	n	Accuracy	Recall	Precision	FPR	Layer 2's real contribution
A - historical, ML-in-middle order	200	1.000	1.000	1.000	0.000	ML caught everything Layer 2 would have needed to
B - reordered, old 4-way judge	200	0.945	0.890	1.000	0.000	Resolved all 100 sent to it; missed 11 attacks (non-destination actions like READ/SUMMARIZE it had no signal for)
C - reordered, batched judge	200	1.000	1.000	1.000	0.000	0 of 100 - all calls failed (model deprecated on this key); ML absorbed everything

Run	n	Accuracy	Recall	Precision	FPR	Layer 2's real contribution
D - reordered, batched judge	1,200	0.998	0.995	1.000	0.000	14 of 600 resolved before quota hit; ML absorbed the other 586
E - reordered, batched judge, full corpus	7,200	0.917	0.889	1.000	0.000	0 of 5,200 - sustained 429 quota errors across every batch; ML absorbed everything

`destination_blind` view throughout. Architecture: Layer 1 (deterministic rule-set agreement) resolves unanimous cases; anything the 3 rule sets split on escalates to Layer 2 (Gemini, batched); anything Gemini can't answer confidently (or fails to reach) escalates to Layer 3 (XGBoost), which is terminal.

Full-corpus funnel (run E): Layer 1 resolved 2,000/7,200 (27.8%), Layer 3 resolved the remaining 5,200 (72.2%).

The honest finding: the cascade's engineering works correctly every time - graceful degradation, no hangs, no crashes, and ML reliably absorbs whatever the LLM layer can't handle. But a real, complete Layer 2 contribution only happened once (run B, on a now-deprecated model), and that run scored *worse* than letting ML handle those cases directly, because the LLM judge is confidently wrong on non-destination attacks (e.g. being talked into reading or summarizing something it shouldn't) in a way the feature-based ML classifier is not. Every other run's strong score came from Layer 1 + Layer 3 alone, with Layer 2 mostly unable to respond due to this API key's quota limits (~20 requests/model/day on the free tier) and, at times, broader Gemini service outages.